Asistente RAG de Inventario (SynerGy): ajustes, validación estadística y repositorio

Repositorio: **https://github.com/Wilixs102/SynerGy-Rag-Inventario** | Modelo: qwen2.5-coder:7b (Ollama) | .NET 10 / SQL Server

1. Ajustes realizados al prototipo

El prototipo convierte preguntas en español sobre el inventario de activos informáticos en consultas T-SQL (text-to-SQL con un LLM local) y las ejecuta sobre la vista *VistaEquipo*. El **baseline** usaba un prompt simple (esquema + códigos de estado). Su error típico fue filtrar el tipo de equipo con *NombreModelo* (el código del fabricante) en lugar de *NombreCategoria/Descripcion*, por lo que contaba mal o devolvía cero filas. Además, las categorías están fragmentadas (p. ej. «COMPUTADOR/Laptop» y «LAPTOP/LAPTOP INCOOP»).

Ajuste	Por qué fue necesario	Mejora esperada
Ingeniería del prompt: reglas de dominio y 5 ejemplos few-shot (ninguno coincide con las preguntas de evaluación)	El modelo no sabía qué columna identifica el tipo de equipo ni la semántica de los estados	Mayor exactitud semántica
Recuperación de pistas de dominio (palabras clave -> filtro correcto) y normalización de la pregunta (minúsculas, sin tildes)	Sinónimos (portátil, celular, teléfono) y categorías duplicadas en la base	Menos errores por vocabulario
Hiperparámetro: temperatura 0.3 -> 0.1	Reducir la aleatoriedad del muestreo	Mayor estabilidad entre repeticiones

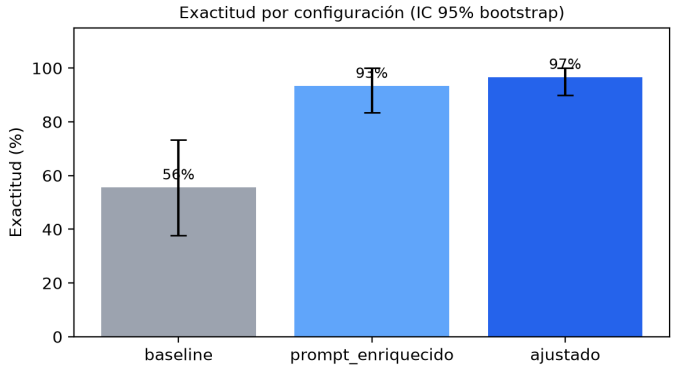
2. Reporte de análisis estadístico

Diseño. 30 preguntas (conteos por estado/tipo/marca, sumas, rankings, fechas, factura y custodio), cada una con criterios verificables: expresiones regulares requeridas y prohibidas sobre el SQL generado. Cada configuración se ejecutó **3 veces** por pregunta (270 generaciones) para medir la variabilidad estocástica del LLM; como no hay entrenamiento, se usa **validación por repeticiones** en lugar de validación cruzada. La unidad independiente es la pregunta, no el intento.

Métricas. Exactitud por componentes (proporción de intentos cuyo SQL cumple todos los criterios), SQL válida (pasa el validador de seguridad), latencia, desviación estándar entre repeticiones y consistencia por pregunta. **Incertidumbre:** IC 95% por bootstrap sobre preguntas (10 000 remuestreos). **Comparación pareada** baseline vs ajustado: prueba de Wilcoxon de rangos con signo (permutación de signos, 100 000) sobre la exactitud por pregunta y prueba de McNemar exacta sobre el resultado por mayoría.

Configuración	Exactitud	IC 95%	DE entre reps	SQL válida	Latencia mediana
baseline (prompt simple, T=0.3)	55.6%	[38%, 73%]	0.019	98.9%	3070 ms
prompt enriquecido (T=0.3)	93.3%	[83%, 100%]	0.000	100.0%	3514 ms
ajustado final (T=0.1)	**96.7%**	[90%, 100%]	0.000	100.0%	3452 ms

Comparación con baseline	Diferencia	IC 95% de la diferencia	Wilcoxon p	McNemar (mejoran/empeoran)	McNemar p
prompt enriquecido	+37.8 pp	[+21.1, +55.6] pp	0.0006	11/0	0.0010
ajustado final	+41.1 pp	[+24.4, +57.8] pp	0.0003	12/0	0.0005



Interpretación. El ajuste elevó la exactitud de 55.6% a 96.7% (+41.1 puntos porcentuales). La mejora es estadísticamente significativa (Wilcoxon p=0.0003; McNemar: 12 preguntas mejoran y 0 empeoran) y el IC 95% de la diferencia no incluye 0. **Variabilidad:** el baseline fue inestable (13 preguntas fallaron siempre y una de forma intermitente); el prompt enriquecido y el ajustado dieron el mismo resultado en las 3 repeticiones (DE = 0), de modo que su incertidumbre proviene del muestreo de preguntas y no del LLM. La diferencia entre *prompt enriquecido* y *ajustado* (93.3% vs 96.7%) corresponde a una sola pregunta y **no es concluyente**: el efecto principal se debe al prompt, no a la temperatura. La latencia no cambió de forma relevante (~3 s).

Limitaciones. (i) 30 preguntas dan ICs amplios; (ii) la exactitud se mide por componentes del SQL (proxy) y no por igualdad de resultados ejecutados contra la base; (iii) las preguntas y criterios fueron escritos por los autores. El único fallo del modelo final (¿Qué marca tiene más equipos?) usó *LIMIT 1*, sintaxis inválida en SQL Server; el validador no detecta dialectos, por lo que «SQL válida» es una métrica de seguridad y no de ejecutabilidad.

3. Consolidación y documentación del repositorio

El repositorio incluye el código final (API .NET 10 con Clean Architecture, cliente Ollama, *RagService*, validador SQL y front mínimo), el harness de evaluación (*tests/SynerGy.RAG.Evaluacion*), los resultados crudos y resumidos (*resultados/*), el script de análisis (*scripts/analizar_resultados.py*), el **README** (descripción, ejecución, dependencias y versiones) y el **CHANGELOG** (bitácora). Evidencia de control de versiones (*git log*):

• cf4afb6 Add .gitattributes, .gitignore, and README.md.

• edd50e6 Add project files.

• be39691 Agrega baseline RAG text-to-SQL con Ollama, validador SQL y front mínimo

• 3cdc106 Ajusta prototipo: prompt con reglas y few-shot, pistas de dominio y temperatura configurable

• 81dabd1 Nombra columnas sin alias en resultados SQL

• 1279a91 Agrega harness de evaluación (30 preguntas, 3 configuraciones) y script de análisis estadístico

• 623cd02 Agrega resultados de evaluación, README y bitácora

4. Síntesis reflexiva

Decisiones técnicas relevantes. (1) Text-to-SQL sobre una única vista en lugar de embeddings: los datos son estructurados y el problema principal era mapear lenguaje a columnas. (2) LLM local (Ollama) para no enviar datos del inventario a terceros. (3) Lista blanca de objetos y validador que solo admite un *SELECT*. (4) Evaluación repetida con estadística pareada, porque el modelo es estocástico y una sola corrida podría engañar.

Riesgo técnico	Mitigación
El SQL generado por el LLM podría modificar datos o leer tablas sensibles (p. ej. Colaborador)	SqlValidator: un único SELECT, sin ; ni comentarios, palabras reservadas bloqueadas y lista blanca (VistaEquipo); usuario de BD de solo lectura
Credenciales expuestas en el repositorio público (cadena de conexión en el primer commit)	Cadena movida a user-secrets/variables de entorno; **pendiente rotar la contraseña** porque permanece en el historial
Alucinaciones o dialecto SQL incorrecto (p. ej. LIMIT)	Prompt con esquema, reglas y ejemplos; evaluación con criterios; siguiente paso: regla TOP y validación sintáctica con SET PARSEONLY
Conjunto de evaluación pequeño y exactitud medida con un proxy	Se reportan ICs bootstrap; trabajo futuro: 100+ preguntas y comparar resultados ejecutados